

A Practical Guide to Gradient boosting

TOPIC -- GRADIENT BOOSTING

-- 01 --

Informal introduction (This section follows the exposition by Cheng Li.) Like other boosting methods, gradient boosting combines weak "learners" into a single strong learner iteratively. It is easiest to explain in the least-squares regression setting, where the goal is to teach a model F to predict values of the form $\hat{y} = F(x)$ by minimizing the mean squared error $\frac{1}{n} \sum_i (\hat{y}_i - y_i)^2$, where i indexes over some training set of size n of actual values of the output variable y :

-- 02 --

Stochastic gradient boosting Soon after the introduction of gradient boosting, Friedman proposed a minor modification to the algorithm, motivated by Breiman's bootstrap aggregation ("bagging") method. Specifically, he proposed that at each iteration of the algorithm, a base learner should be fit on a subsample of the training set drawn at random without replacement. Friedman observed a substantial improvement in gradient boosting's accuracy with this modification. Subsample size is some constant fraction f of the size of the training set. When $f = 1$, the algorithm is deterministic and identical to the one described above. Smaller values of f introduce randomness into the algorithm and help prevent overfitting, acting as a kind of regularization. The algorithm also becomes faster, because regression trees have to be fit to smaller datasets at each iteration. Friedman obtained that $0.5 \leq f \leq 0.8$ leads to good results for small and moderate sized training sets. Therefore, f is typically set to 0.5, meaning that one half of the training set is used to build each base learner. Also, like in bagging, subsampling allows one to define an out-of-bag error of the prediction performance improvement by evaluating predictions on those observations which were not used in the building of the next base learner. Out-of-bag estimates help avoid the need for an independent validation dataset, but often underestimate actual performance improvement and the optimal number of iterations.

-- 03 --

$$F_m(x) = F_{m-1}(x) - \gamma_m \sum_{i=1}^n \nabla F_{m-1}(y_i, F_{m-1}(x_i))$$

$$\{ \displaystyle F_{\{m\}}(x) = F_{\{m-1\}}(x) - \gamma_{\{m\}} \sum_{\{i=1\}^{\{n\}} \{ \nabla_{\{F_{\{m-1\}}\}}(y_{\{i\}}, F_{\{m-1\}}(x_{\{i\}})) \} \}$$

-- 04 --

Gradient boosting is a machine learning technique based on boosting in a functional space, where the target is pseudo-residuals instead of residuals as in traditional boosting. It gives a prediction model in the form of an ensemble of weak prediction models, i.e., models that make very few assumptions about the data, which are typically simple decision trees. When a decision tree is the weak learner, the resulting algorithm is called gradient-boosted trees; it usually outperforms random forest. As with other boosting methods, a gradient-boosted trees model is built in stages, but it generalizes the other methods by allowing optimization of an arbitrary differentiable loss function.

-- 05 --

$$F_0(x) = \arg \min_{h_m \in H} \sum_{i=1}^n L(y_i, h_m(x_i)),$$

$$F_{\{0\}}(x) = \{ \underset{\{h_{\{m\}} \in \{ \mathcal{H} \} \}}{\arg \min} \} \sum_{\{i=1\}^{\{n\}} \{ L(y_{\{i\}}, h_{\{m\}}(x_{\{i\}})) \} \}$$

-- 06 --

$$F_m(x) = F_{m-1}(x) + (\arg \min_{h_m \in H} [\sum_{i=1}^n L(y_i, F_{m-1}(x_i) + h_m(x_i))])(x),$$

$$\{ \displaystyle F_{\{m\}}(x) = F_{\{m-1\}}(x) + \left(\{ \underset{\{h_{\{m\}} \in \{ \mathcal{H} \} \}}{\operatorname{arg\,min}} \} \left[\sum_{\{i=1\}^{\{n\}} L(y_{\{i\}}, F_{\{m-1\}}(x_{\{i\}}) + h_{\{m\}}(x_{\{i\}})) \right] \right) (x), \}$$

-- 07 --

where parameter ν $\{ \displaystyle \nu \}$ is called the "learning rate". Empirically, it has been found that using small learning rates (such as $\nu < 0.1$ $\{ \displaystyle \nu < 0.1 \}$) yields dramatic improvements in models' generalization ability over gradient boosting without shrinking ($\nu = 1$ $\{ \displaystyle \nu = 1 \}$). However, it comes at the price of increasing computational time both during training and querying: lower learning rate requires more iterations.

-- 08 --

$$-\partial L_{MSE} \partial F(x_i) = 2n(y_i - F(x_i)) = 2nh_m(x_i).$$

$$\{ \partial L_{\{rm \{MSE\}\}} \partial F(x_{\{i\}}) = \{ \frac{2}{\{n\}} \} (y_{\{i\}} - F(x_{\{i\}})) = \{ \frac{2}{\{n\}} \} h_{\{m\}}(x_{\{i\}}) \}$$

-- 09 --

where $\gamma > 0$. For small γ , this implies that $L(y_i, F_m(x_i)) \leq L(y_i, F_{m-1}(x_i))$.
 $L(y_{\{i\}}, F_{\{m\}}(x_{\{i\}})) \leq L(y_{\{i\}}, F_{\{m-1\}}(x_{\{i\}}))$.

-- 10 --

$\hat{F} = \operatorname{argmin}_{F \in \mathbb{E}_{x,y}[L(y, F(x))]} \mathbb{E}_{x,y}[L(y, F(x))]$.

-- 11 --

In the discrete case however, i.e. when the set H is finite, we choose the candidate function h closest to the gradient of L for which the coefficient γ may then be calculated with the aid of line search on the above equations. Note that this approach is a heuristic and therefore doesn't yield an exact solution to the given problem, but rather an approximation. In pseudocode, the generic gradient boosting method is:

-- 12 --

Number of observations in leaves Gradient tree boosting implementations often also use regularization by limiting the minimum number of observations in trees' terminal nodes. It is used in the tree building process by ignoring any splits that lead to nodes containing fewer than this number of training set instances. Imposing this limit helps to reduce variance in predictions at leaves.

-- 13 --

When the loss $L(\cdot, \cdot)$ is mean-squared error (MSE) the coefficients γ_j coincide with the coefficients of the tree-fitting procedure b_j .

-- 14 --

$F_m(x) = F_{m-1}(x) + \nu \cdot \gamma_m h_m(x)$, $0 < \nu \leq 1$,
 $F_{\{m\}}(x) = F_{\{m-1\}}(x) + \nu \cdot \gamma_{\{m\}} h_{\{m\}}(x)$, $0 < \nu \leq 1$,